Domestic Flight Price Prediction: Regression Report

Oct 15, 2026 · @Yug Sahni

1. Problem and dataset

The task is to predict the ticket price (a continuous value) of a domestic flight from its booking and itinerary details. It is the regression half of the course project; the Kaggle metric appears to be R^2 (the competition page needs a login, so this was not confirmed from the page itself).

File	Rows	Notes
train.csv	15,589	11 features plus the target `price`
test.csv	3,898	same features, no target
sample_submission.csv	3,898	columns `id`, `price`

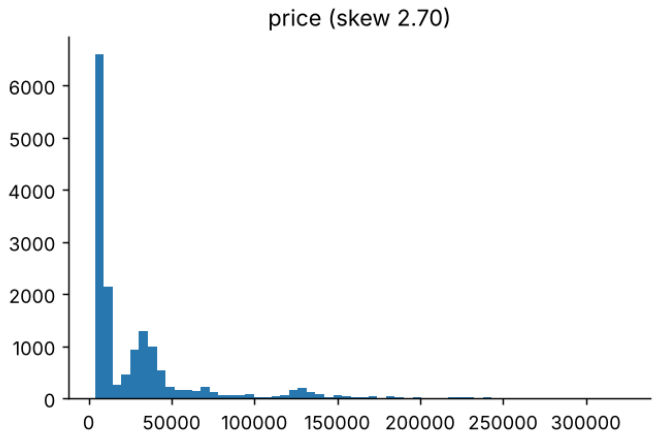
Features: `airline`, `flight` (code plus number, e.g. `AI 441`), `source_city`, `destination_city`, `departure_time`, `arrival_time`, `stops` (zero / one / two_or_more), `flight_class` (Economy / Business), `duration` (hours) and `days_left` (days before departure). Prices range from 3,801 to 323,177 with a mean of 29,817 and a median of 10,183.

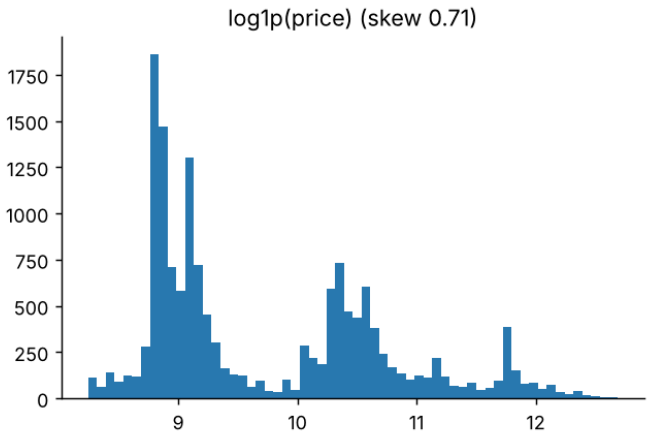
Only models taught in Part 1 are used.

2. Exploratory data analysis

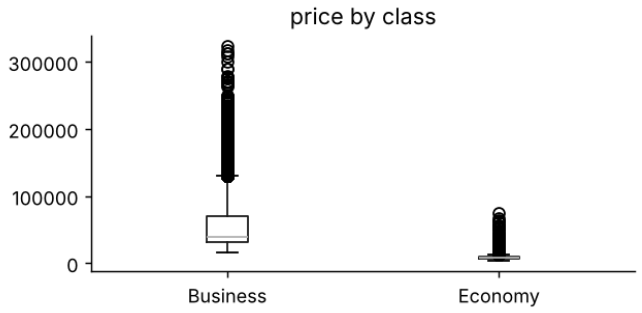
Two columns have missing values, and both gaps are recoverable. `airline` is missing in 19.8% of train rows and `duration` in 19.9%; test has the same rates (20.5% and 19.9%). Missingness is unrelated to price (mean price 29,845 vs 29,701 for duration; 29,685 vs 30,348 for airline).

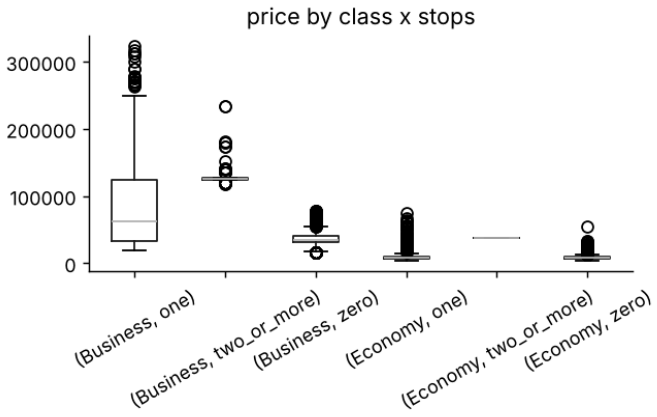
The target is heavily right-skewed (skew 2.70, falling to 0.71 after a log transform).



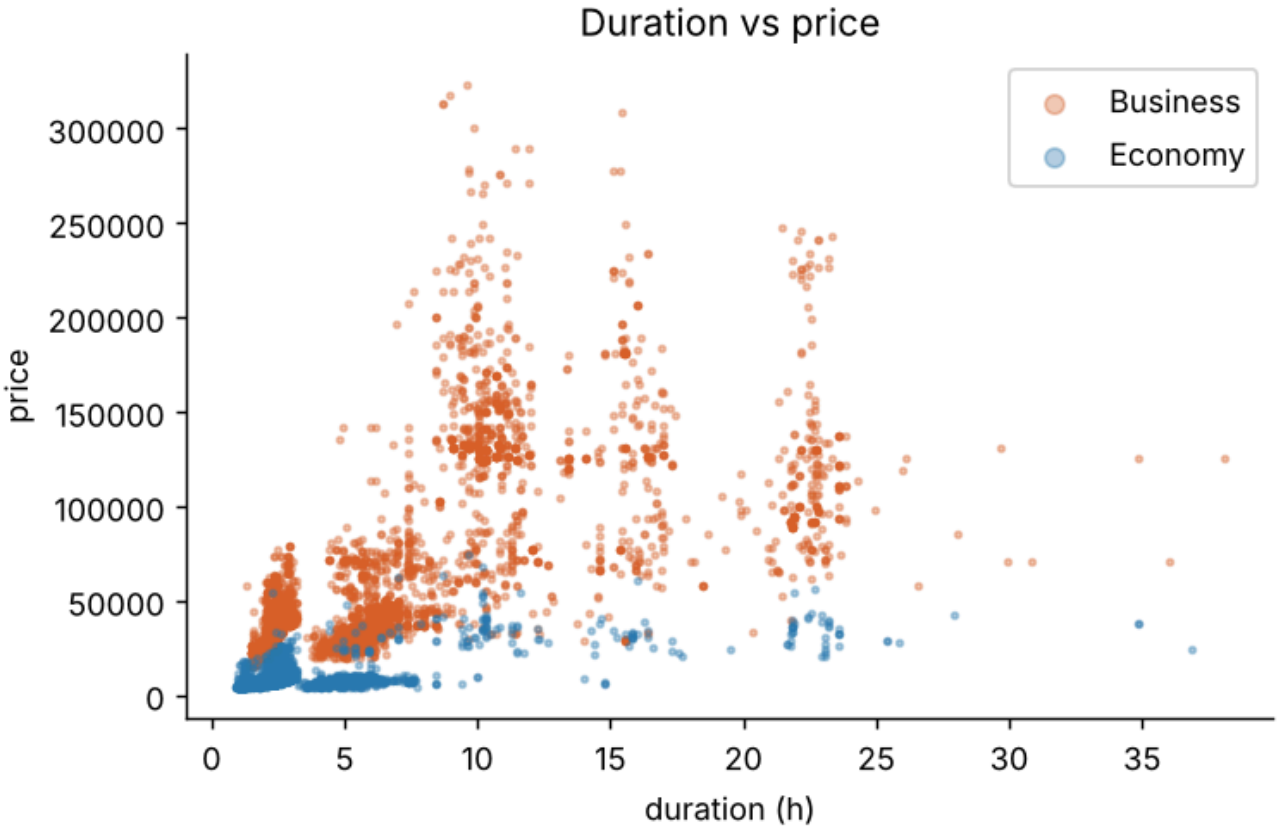


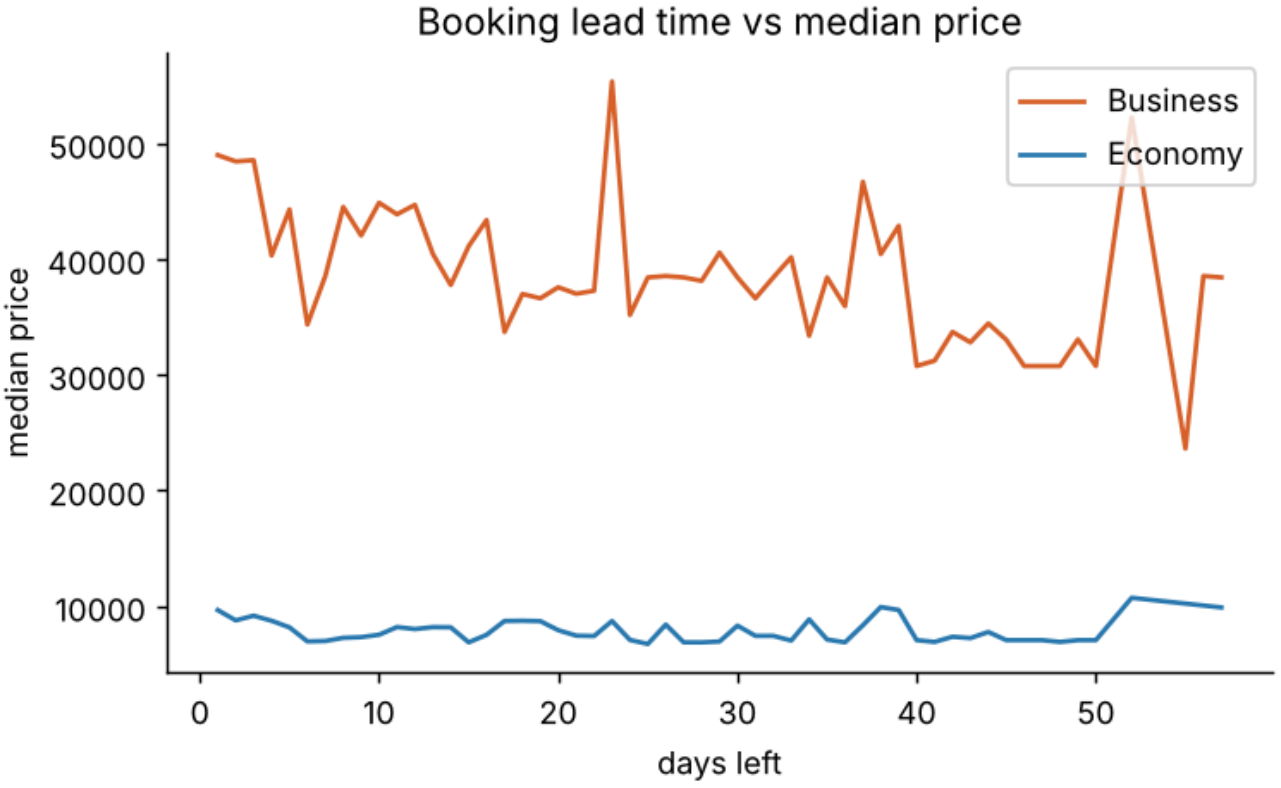
Class and stops dominate price. Economy has a median of 7,587 against 38,546 for Business. Business with one stop is the most expensive and the most variable group.





Duration is the strongest numeric correlate of price (r = 0.70), followed by stops (0.57) and class (0.65). Booking lead time is nearly uncorrelated (r = −0.02), apart from a surge for bookings within about 5 days of departure.





Four more observations shaped the modelling:

- **Airline is a function of the flight code.** Each prefix maps to exactly one airline (`6E` is Oceanic Airlines, `EY` is Nakatomi Airlines), so every missing airline can be recovered.

- **Flight identity explains most of the price.** Flight plus class alone explains 96.5% of the variance of log price. 98.8% of train rows share their flight and class with at least one other row, and only 0.9% of test flights never appear in train.
- **Prices are noisy within a flight.** The same flight and route can show prices from 81k to 313k (for example `EY 353` Business). About 65% of the unexplained error comes from the top 1% of rows, almost all expensive Business one-stop tickets.
- **There are exact duplicates.** 314 train rows repeat another row's features, and every such group shares an identical price. About 8% of test rows have an exact twin in train.

Other plots (missingness, airline and route prices, correlations) are in `outputs/figures/` in the code repository.

3. Data preprocessing and justification

Step	What was done	Why
Airline imputation	Filled from the flight-code prefix	The prefix-to-airline map is one-to-one, so all 3,089 missing values are recovered exactly, with no guessing
Duration imputation	Group median over the most specific matching key: flight + route + stops + departure/arrival time, then coarser keys down to route + stops	The same flight and route repeats, so durations are recoverable. On 2,000 known durations masked at random the imputer was exactly right for 91% and had R^2 of 0.96
Missing flags	Tested, then dropped	Missingness has no relation to price
Flight code and number	Split `flight` into carrier code and numeric flight number	The code is a clean low-cardinality categorical
Ordinal encodings	`stops` to 0/1/2, class to a 0/1 flag, time-of-day to an ordered 0–5 scale	These variables have a natural order that trees and the linear model can use
Route and class keys	Added `route`, `flight	class `and` route
Late-booking flag	`days_left <= 5`	EDA shows a price surge for the last few days

Step	What was done	Why
Categorical encoding	One-hot for the linear model; ordinal codes plus out-of-fold target encoding for Decision Tree, Random Forest and XGBoost; CatBoost's own categorical handling	One-hot on about 2,000 flight keys would be huge, so those are target-encoded. Encoding is fitted inside each CV fold to avoid leakage
Target transform	Raw price vs `log1p(price)`, chosen per model by CV	The log removes the skew. Scores are always computed on the raw price, because that is what the metric sees
Scaling	Standardised numerics for the linear model only	Tree models do not need it

Imputation statistics use only feature columns (never price) and are computed over train and test together. This is label-free and cannot leak the target.

4. Which Part 1 models apply to a continuous target

Seven models give a numeric prediction and were all tried: Linear Regression, Ridge, Lasso, Decision Tree, Random Forest, XGBoost and CatBoost. The classifiers do not apply.

Topic	Used?	Reason
Linear Regression	Yes	Baseline regression model
Ridge Regression	Yes	Linear regression with an L2 penalty on the coefficients (the maximum a posteriori estimate under a Gaussian prior)
Lasso Regression	Yes	Linear regression with an L1 penalty, which can set coefficients to exactly zero (a Laplace prior)
Maximum Likelihood Estimation	Not a model	It is an estimation principle. With Gaussian noise, maximising the likelihood is the same as minimising squared error, which is how Linear Regression is fitted. It justifies the linear models rather than being a separate entry
Decision Tree	Yes	Regression tree that splits by variance reduction

Topic	Used?	Reason
Random Forest	Yes	Average of many regression trees
XGBoost	Yes	Gradient-boosted regression trees
CatBoost	Yes	Gradient-boosted trees with native categorical handling
Bayes Classifier	No	Picks the class with the highest posterior P(class given x), so it needs a finite set of classes
Naive Bayes	No	Same reason: it models P(x given class) per class and outputs a class label
Logistic Regression	No	Despite the name it is a classifier. It models a class probability through a sigmoid and is trained with cross-entropy loss, so it outputs a class, not a price

Binning the price into classes would make the three classifiers usable, but predictions would then be bin values and R^2 would drop sharply, so this was not done.

5. Models used and best score

Scores are R^2 on the raw price. "CV" is the best tuned 5-fold score on 80% of train (with standard deviation across folds). "Holdout" is the 20% never used for tuning. "OOF" is the 5-fold out-of-fold score over all of train.

Model	Best CV R^2	Holdout R^2	OOF R^2	Kaggle public score
Linear Regression	0.8562 ± 0.0102	0.8621	0.8560	0.86926
Ridge Regression	0.8562 ± 0.0102	0.8621	0.8560	0.86926
Lasso Regression	0.8562 ± 0.0102	0.8622	0.8559	0.86928
Decision Tree	0.9059 ± 0.0114	0.9104	0.9077	0.94039
Random Forest	0.9269 ± 0.0081	0.9334	0.9274	0.95937

Model	Best CV R^2	Holdout R^2	OOF R^2	Kaggle public score
XGBoost	0.9280 ± 0.0113	0.9290	0.9287	0.95820
CatBoost	0.9276 ± 0.0061	0.9296	0.9282	0.95419
Extra: average of Random Forest, XGBoost and CatBoost	not tuned	not scored	0.9312	0.96080
Extra: that average, with exact-duplicate lookup	not tuned	not scored	0.9330	**0.96147**

The Kaggle column holds the public leaderboard scores of the submission files in `outputs/submissions/` (the public part is about 60% of the test rows). All were submitted within a few minutes of each other.

Key observation per model:

- **Linear, Ridge and Lasso:** all three tie at 0.856. The data has far more rows than features, so the penalty has almost nothing to shrink (the best alpha is tiny) and Lasso keeps essentially every coefficient. These are the weakest models because price depends on the flight and class in a strongly non-additive way (Business one-stop prices are many times Economy ones), which no linear model can represent.
- **Decision Tree:** a large jump to 0.906. Even one tree captures the flight, class and stops structure, but it is high variance.
- **Random Forest:** averaging trees removes most of that variance (+0.021).
- **XGBoost:** the best single model on CV and OOF, with the largest spread across folds.
- **CatBoost:** statistically tied with XGBoost, and the most stable across folds (standard deviation 0.006).
- **Average ensemble:** the three tree ensembles make partly different errors, so their plain average gains about 0.002 out-of-fold.
- **Exact-duplicate lookup:** 307 test rows have a feature-identical train row, and the price is always identical in train for such groups. Using that price gains about 0.002 out-of-fold. It is not one of the Part 1 models, so it is shown separately.

Models in the team notebook (a separate approach, not yet submitted). The same models were also run in a Colab notebook (`Untitled7.ipynb`) on the 15,310 training rows left after removing 279 exact duplicates. The imputation sits inside each pipeline, so it is fitted on the training fold only. Scores are R^2 from 5-fold cross-validation repeated 3 times; the "train" column shows the gap that signals overfitting.

Model	Best CV R^2	Train R^2	Kaggle public score
Baseline: predict the mean	−0.0004	0.0000	not submitted
Linear Regression	0.8597 ± 0.0082	0.8621	not submitted
Ridge Regression (alpha 1)	0.8594 ± 0.0080	0.8619	not submitted
Lasso Regression (alpha 10)	0.8585 ± 0.0081	0.8611	not submitted
Ridge on log(price)	0.8419 ± 0.0113	0.8449	not submitted
Polynomial (degree 2) + Ridge (alpha 3)	0.9189 ± 0.0052	0.9327	not submitted
Decision Tree (depth 10, min leaf 5)	0.9111 ± 0.0083	0.9351	not submitted
Random Forest (200 trees, depth 14, min leaf 5)	0.9230 ± 0.0070	0.9522	not submitted
Gradient Boosting (400 iterations, depth 6)	0.9257 ± 0.0069	0.9451	not submitted
Ensemble: Poly-Ridge (weight 1) + Gradient Boosting (weight 2)	0.9270 ± 0.0062	0.9432	not submitted

An unlimited-depth tree scored 0.8707 on CV against 0.9993 on train, which shows what real overfitting looks like. Adding polynomial terms to Ridge lifts it from 0.859 to 0.919, because the interactions (class × stops × duration) matter. The ensemble of the two different model types is the best by a small margin.

6. Hyperparameter tuning

Protocol (the same for every model). 20% of train is held out with seed 42 and never touched during tuning. Hyperparameters are chosen by 5-fold cross-validation (shuffled, seed 42) on the other 80%, scored by R^2 on the raw price. The best setting is then refit on the 80% and scored once on the holdout, giving an honest estimate. Finally it is refit on all of train to predict test. Whether to train on the raw or the log price is treated as one more hyperparameter.

Model	Search	Trials	Best setting
Linear Regression	Full grid: log target yes/no, class interactions yes/no	4	log target, no interactions (CV 0.8562; raw target 0.8500; interactions hurt, 0.8485 and 0.8452)
Ridge Regression	Random: alpha 0.001–1000 (log scale), log target, interactions	30	alpha 0.016, log target, no interactions
Lasso Regression	Random: alpha 1e-6 to 0.1 (log scale), log target, interactions	20	alpha 1e-5, log target, no interactions
Decision Tree	Random: depth 3–25, min leaf 1–29, min split 2–39, cost-complexity alpha 0.001–1000, max features, target-encoding smoothing, log target	60	depth 15, min leaf 29, min split 16, alpha 0.042, raw target
Random Forest	Random: trees 200/400, depth, min leaf 1–7, max features 0.3–1.0, bootstrap, smoothing, log target	24	200 trees, depth 20, max features 0.7, min leaf 1, bootstrap, log target
XGBoost	Random: trees 200–1200, learning rate 0.02–0.2, depth 3–10, min child weight 1–11, subsample 0.6–1.0, column sample 0.5–1.0, L2 0.1–30, L1 0.001–5, gamma 0.001–5, smoothing, log target	45	940 trees, learning rate 0.054, depth 9, min child weight 10, subsample 0.77, column sample 0.52, L2 0.16, L1 0.005, gamma 0.03, log target
CatBoost	Random: iterations 600/1000/1500, learning rate 0.03–0.25, depth 4–9, L2 1–20, random strength 0.1–5, bagging temperature 0–2, borders 128/254, log target	16	1000 iterations, learning rate 0.139, depth 6, L2 1.18, random strength 0.23, bagging temperature 1.73, 254 borders, log target

Three things the search showed:

- The log target helped every model except the Decision Tree (best CV with log vs raw: Random Forest 0.9269 vs 0.9254, XGBoost 0.9280 vs 0.9268, CatBoost 0.9276 vs 0.9254, Decision Tree 0.8782 vs 0.9059).
- Untuned or badly tuned trees are far worse than tuned ones. Decision Tree trials ranged from −0.11 to 0.906 and XGBoost trials from 0.866 to 0.928.

- The tuned boosted models and the forest end within 0.001 of each other, which is inside the fold-to-fold noise (standard deviation 0.006–0.011). More search is unlikely to change the ranking.

Every trial is logged in `outputs/tuning_<model>.csv` in the code repository. CatBoost had the fewest trials because each fit is slow on the available 2 CPU cores.

Tuning in the team notebook. The notebook tuned by hand with repeated 5-fold CV: Ridge alpha 1, 3, 10, 30, 100 on polynomial features (best 3 to 10); tree depth 6, 8, 10, 12 crossed with minimum leaf 5, 10, 20 (best depth 10, leaf 5); and fixed settings for the Random Forest (200 trees, depth 14, leaf 5) and the gradient-boosting model (400 iterations, learning rate 0.05, depth 6, leaf 20, L2 1.0). The final ensemble weights (1 : 2) were chosen by comparing CV scores.

7. Results and discussion

Where the models stand. Tree ensembles reach about 0.93, a single tree 0.91 and the three linear models (plain, Ridge, Lasso) 0.86. Gradient boosting did not clearly beat the Random Forest here: the gaps among the three ensembles (under 0.002) are smaller than the fold-to-fold noise.

Public leaderboard versus cross-validation. Every model scored higher on the public part than in cross-validation: the linear models by about 0.013 (0.856 to 0.869), the Decision Tree by 0.033 (0.908 to 0.940), Random Forest by 0.032, XGBoost by 0.029, CatBoost by 0.026 and the average ensemble by 0.030 (0.931 to 0.961). The tree models gain more than the linear ones, so the public rows are easier for models that fit the flight-level structure, most likely because fewer very expensive Business one-stop tickets fall in them. The order among the three tree ensembles on the public part (Random Forest 0.959, XGBoost 0.958, CatBoost 0.954) differs slightly from the cross-validation order, but the gaps are within the noise measured below.

Why about 0.93 looks like the ceiling for these features. Most of the remaining error is irreducible noise, concentrated in a few expensive tickets. About 2% of rows (Business, one stop, premium carriers) produce roughly two thirds of the squared error. Within one flight and class (for example EY 353 Business) the price still ranges from 73k to 313k, even for the same route and duration, and no available column explains it (row `id`, booking lead time and imputed duration were all checked). Predicting each row from the mean price of the other rows with the same flight, route and class scores only 0.92, and the tuned models already beat that. Adding exact-itinerary keys to XGBoost did not help (0.9280 to 0.9267).

Can we beat the top public score of 0.96427? The best public score so far is 0.96147 (the average ensemble with the exact-duplicate lookup), only 0.0028 below it. Cross-validation had put our best model near 0.931, so the public score came out far higher than my estimate. That gap is the useful finding: the public rows are evidently easier than a random draw from train, most likely because fewer of the very expensive Business one-stop tickets landed in them. A gap of 0.0028 is about the size of the noise between two similar models (section below), so overtaking 0.96427 is possible, but by chance more than by design. I found no modelling change worth more than about 0.002.

How much of a public score is luck? I simulated the leaderboard: repeatedly draw 3,898 rows of out-of-fold predictions, score a random 60% (2,339 rows) as the public part and the other 40% (1,559 rows) as the private part, 5,000 times.

Model	Mean public R^2	Public SD	90% range of public R^2	Private SD	SD of (public − private)
XGBoost	0.929	0.0090	0.913 to 0.943	0.0116	0.0154
Average ensemble	0.931	0.0086	0.917 to 0.945	0.0111	0.0147

- A model's public score has a luck component of about ±0.009 (one standard deviation) in this simulation, and its private score about ±0.012. The difference between the two moves by about ±0.015.
- The simulation draws rows from train, so it cannot see how the real test set differs. The observed 0.96080 (average ensemble) is about 3.4 standard deviations above its mean, which tells us the real public part is easier than a typical draw. Treat the simulated spread as a lower bound on the luck involved.
- A gap of 0.001–0.003 between two leaderboard entries is noise. For our two best models, the better one on the public part is the better one on the private part only about 80% of the time (the ranking flips in 20% of draws).
- The private part is a smaller sample (about 1,559 rows) and may contain a different share of expensive tickets, so the private score can differ from 0.96080 by a few hundredths in either direction. Do not read the public score as a forecast of the final one.
- Pick the final submission by cross-validation, not by the public score.

8. Best model

By cross-validation the best single model is **XGBoost** (CV R^2 0.9280, OOF 0.9287), with CatBoost (0.9276) and Random Forest (0.9269) statistically tied, and the best overall predictor is the **equal-weight average of Random Forest, XGBoost and CatBoost** (OOF

R^2 0.9312, 0.9330 with the exact-duplicate lookup).

Best public leaderboard score: 0.96147, from `submission_average_ensemble_twin_override.csv`: the average of tuned Random Forest (200 trees), XGBoost (940 trees) and CatBoost (1000 iterations), with the price copied from the training set whenever a test row exactly repeats a training row (307 test rows). The same average without the lookup scored 0.96080, so the lookup added 0.00067. Among the Part 1 models alone, the best are the average ensemble (0.96080) and the Random Forest (0.95937). The lookup is not a Part 1 model, so if only Part 1 models count, report the average ensemble as the best model. The public order among these top entries is within noise (section 7), so the final choice is made by cross-validation, not by the public score.

9. GitHub link and reproducibility

- **Seeds:** one constant, `SEED = 42` in `src/config.py`, drives the CV splits, the holdout split, NumPy, and every model's `random_state` or `seed`. Refitting XGBoost from the saved settings reproduced the saved test predictions to within 0.01 in price.
- **GitHub link:** add the repository link here. The repository must contain the notebook `Untitled7.ipynb` (the team's second approach) as well as the `src/` code (it generated every submission scored in section 5). The code has not been pushed yet.
- **Layout:** `src/features.py` (imputation and features), `src/models.py` (models and search spaces), `src/train.py` (tuning and evaluation), `src/eda.py`, `src/luck.py` (leaderboard noise), `src/ensemble.py`, `src/twin_override.py`, `run_all.sh` (runs everything).
- **Libraries:** scikit-learn 1.9.1, XGBoost 3.4.1, CatBoost 1.2.10.